

08

CAPÍTULO

Framework de Engenharia UNIIKER (UEF)

*Como transformar uma ideia em produto funcional e escalável:
metodologia proprietária em 15 etapas, da concepção à operação.*

Este capítulo apresenta o **UNIIKER Engineering Framework (UEF)**, metodologia proprietária que estrutura o ciclo de vida completo de soluções inteligentes: engenharia de produto, requisitos, arquitetura, escolha de plataforma, sensores, protocolos, IA, testes, implantação, operação, manutenção e escalabilidade. Quatro novas caixas editoriais — Checklist do Engenheiro, Critério de Aprovação, Erros Clássicos e Visão do Consultor — orientam a transformação de protótipos em produtos profissionais.

UEF

METODOLOGIA

ENGENHARIA

CICLO DE VIDA

ISO 15288

PMBOK

Sumário do Capítulo

8.1 Engenharia de Produto	3
8.1.1 As Cinco Fases	3
8.1.2 Por que muitos projetos morrem antes da produção?	4
8.2 O Framework UNIIKER (UEF)	4
8.2.1 As 15 Etapas do UEF	5
8.3 Engenharia de Requisitos	6
8.3.1 Requisitos Funcionais e Não Funcionais	6
8.3.2 Restrições, Premissas, Stakeholders e Critérios	7
8.4 Escolha da Plataforma	7
8.4.1 Metodologia de Escolha	8
8.5 Seleção de Sensores	9
8.6 Comunicação	9
8.7 Arquitetura	10
8.7.1 Checklist Arquitetural	11
8.8 Inteligência Artificial	11
8.9 Testes	12
8.9.1 Níveis de Teste	12
8.10 Implantação	13
8.10.1 Piloto e Rollout	13
8.11 Operação	14
8.11.1 Componentes de Operação	14
8.12 Manutenção	14
8.13 Escalabilidade	15
8.14 Estudos de Caso	16
8.14.1 ETA (Estação de Tratamento de Água)	16
8.14.2 ETE (Estação de Tratamento de Esgoto)	16
8.14.3 Agricultura de Precisão	16
8.14.4 Hospital Inteligente	16

8.14.5 Smart City	16
8.14.6 Universidade	16
8.14.7 Amazônia	17
8.15 Checklist Final	17
8.15.1 Matriz RACI	19
Conclusão do Capítulo	19
Referências Bibliográficas	20

CAPÍTULO 8

Framework de Engenharia UNIHAKER (UEF)

Metodologia proprietária para transformar ideias em produtos funcionais e escaláveis

Após consolidar o conhecimento de hardware (Capítulos 3-4), escolha de plataforma (Capítulo 5), arquitetura (Capítulo 6) e catálogo de projetos (Capítulo 7), este capítulo apresenta o UNIHAKER Engineering Framework (UEF) — metodologia proprietária em 15 etapas que estrutura o ciclo de vida completo de soluções inteligentes, da concepção à operação em produção. Quatro novas caixas editoriais — Checklist do Engenheiro, Critério de Aprovação, Erros Clássicos e Visão do Consultor — orientam a transformação de protótipos em produtos profissionais.

8.1 Engenharia de Produto

Engenharia de produto é a disciplina que transforma uma necessidade de mercado em uma solução técnica viável, escalável e comercialmente sustentável. No contexto de sistemas inteligentes baseados em UNIHAKER, engenharia de produto articula hardware, software, comunicação, inteligência artificial, operação e manutenção em um ciclo de vida coerente. Esta seção analisa as cinco fases fundamentais desse ciclo: ideia, protótipo, MVP, piloto e produto comercial.



Figura 8.1 — Ciclo de vida do produto: da Ideia ao Produto Comercial, em cinco fases. Risco diminui e custo de mudança aumenta ao longo do ciclo. Fonte: elaborado pelo autor.

8.1.1 As Cinco Fases

Ideia é a fase conceitual, no papel, sem código ou hardware. Identifica-se um problema, propõe-se uma solução e justifica-se sua relevância. Custo: baixo (semanas de trabalho intelectual). Risco: alto (a ideia pode ser inválida). **Protótipo** é a validação técnica: prova-se que a tecnologia funciona no laboratório, sem preocupação com robustez, escala ou usabilidade. Custo: baixo-médio (USD 100-1.000 em hardware). Risco: médio (a tecnologia pode falhar em condições reais). **MVP** (Minimum Viable Product) é a primeira versão utilizável por stakeholders reais, com funcionalidades mínimas mas completas. Custo: médio (USD 1.000-10.000). Risco: médio-alto (pode não atender necessidades reais).

Piloto é a implantação controlada em ambiente real, com poucos usuários, coleta de feedback estruturado e iteração. Custo: médio-alto (USD 10.000-100.000). Risco: médio (pode haver problemas operacionais). **Produto Comercial** é a versão final, escalável, documentada, com suporte, manutenção e plano de evolução. Custo: alto (USD 100.000+). Risco: baixo em tecnologia, mas alto em mercado (pode não vender).

8.1.2 Por que muitos projetos morrem antes da produção?

Estatísticas de incubadoras e aceleradoras mostram que 60-80% dos projetos de hardware IoT morrem entre o protótipo e a produção comercial. As causas são recorrentes: (i) protótipo funciona no laboratório mas falha em ambiente real (temperatura, umidade, EMI); (ii) custo de produção supera preço que mercado paga; (iii) ausência de plano de manutenção; (iv) equipe técnica não consegue escalar para milhares de dispositivos; (v) regulamentações não atendidas (ANATEL, ANVISA, CE/FCC); (vi) parceiros de fabricação não entregam em prazo/qualidade; (vii) burnout da equipe após 18+ meses de desenvolvimento.

▲ ERROS CLÁSSICOS

Erro Clássico #1: «Funciona no laboratório, falha no campo.»

Causa: testes limitados a condições ideais (temperatura 25°C, umidade 50%, sem EMI).

Como evitar: testes climáticos (câmara térmica -20 a +70°C), testes de EMI (câmara anecóica), testes de campo por 90+ dias antes do rollout. Orçamento mínimo para testes: 15-20% do custo total do projeto.

Exemplo real: startup de irrigação smart que funcionou em laboratório, mas falhou em campo: sensores de umidade do solo foram corroídos por fertilizantes em 3 meses. Causa: ausência de testes de imersão em fertilizantes. Solução: encapsulamento hermético IP68.

▲ VISÃO DO CONSULTOR

Visão do Consultor: uma empresa de venture capital avalia um projeto IoT em três dimensões críticas: (1) **Tecnologia:** tem defesa técnica (patentes, know-how)?; (2) **Mercado:** TAM (Total Addressable Market) acima de USD 100M?; (3) **Equipe:** tem competência para escalar de 100 para 100.000 dispositivos? Projetos que falham em qualquer dimensão não recebem investimento série A. Recomendação: antes de buscar investimento, validar as três dimensões com consultoria externa.

8.2 O Framework UNIHAKER (UEF)

O **UNIHAKER Engineering Framework (UEF)** é uma metodologia proprietária que estrutura o ciclo de vida completo de soluções inteligentes em 15 etapas sequenciais. Inspirado em padrões internacionais (ISO 15288, ISO 25010, PMBOK, INCOSE Handbook), o UEF adapta esses frameworks genéricos à realidade

específica de projetos baseados em plataformas UNIIKER K10 e M10, integrando Edge AI, IoT e computação distribuída.

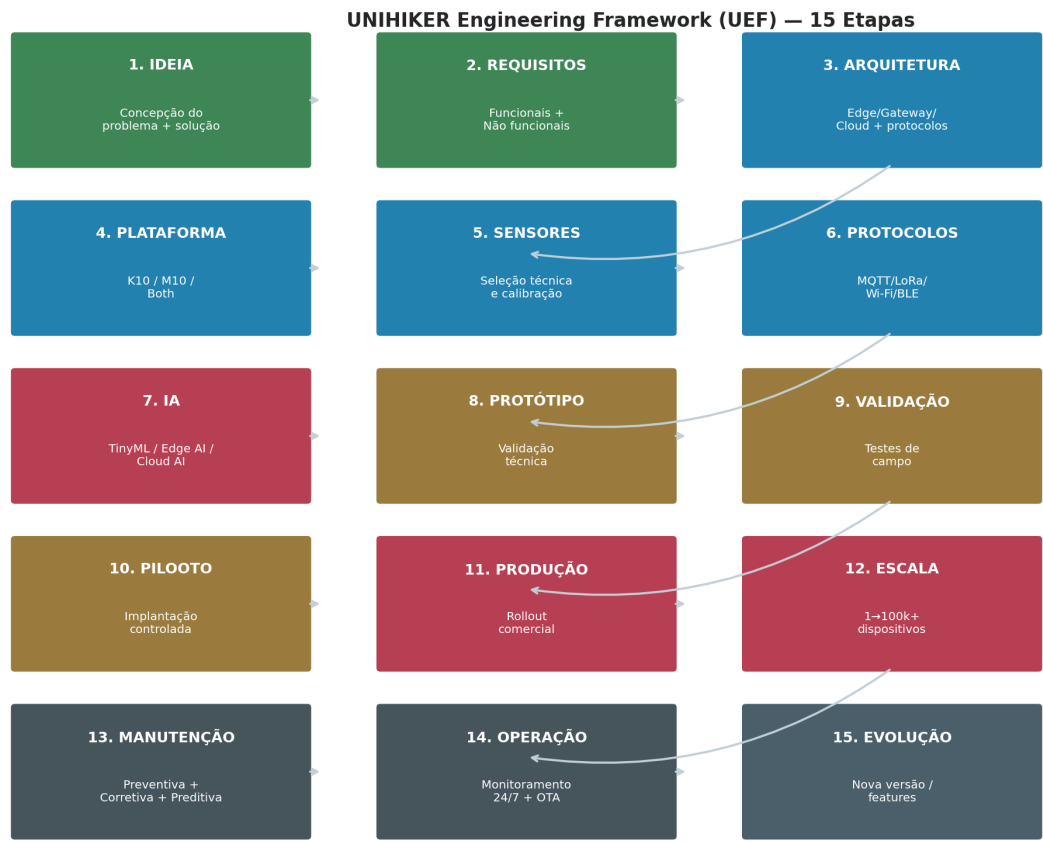


Figura 8.2 — UNIIKER Engineering Framework (UEF) em 15 etapas sequenciais, da Ideia à Evolução. Cada etapa produz artefatos que alimentam a etapa seguinte. Fonte: elaborado pelo autor.

8.2.1 As 15 Etapas do UEF

O UEF compreende 15 etapas agrupadas em quatro macro-fases: **Concepção** (etapas 1-3: Ideia, Requisitos, Arquitetura), **Design Técnico** (etapas 4-7: Plataforma, Sensores, Protocolos, IA), **Implementação** (etapas 8-11: Protótipo, Validação, Piloto, Produção) e **Operação** (etapas 12-15: Escala, Manutenção, Operação, Evolução). Cada macro-fase tem entregáveis específicos, checkpoints de qualidade e critérios de aprovação para avançar à macro-fase seguinte.

#	Etapa	Macro-fase	Entregável Principal	Checkpoint
1	Ideia	Concepção	Canvas do projeto	Stakeholders alinhados
2	Requisitos	Concepção	Especificação formal	Cliente aprova
3	Arquitetura	Concepção	Diagrama arquitetural	Revisão técnica

#	Etapa	Macro-fase	Entregável Principal	Checkpoint
4	Plataforma	Design	K10/M10/Both justificado	Trade-off documentado
5	Sensores	Design	Lista BOM com calibração	MTBF estimado
6	Protocolos	Design	Stack de comunicação	Latência validada
7	IA	Design	Modelo + dataset	Acurácia medida
8	Protótipo	Implementação	Hardware funcional	Demo ao cliente
9	Validação	Implementação	Relatório de testes	Testes passam
10	Piloto	Implementação	Implantação controlada	90 dias estável
11	Produção	Implementação	Rollout comercial	Suporte ativo
12	Escala	Operação	1k+ dispositivos	Custo sob controle
13	Manutenção	Operação	Plano preventivo	MTTR medido
14	Operação	Operação	Monitoramento 24/7	SLA cumprido
15	Evolução	Operação	Roadmap v2	Roadmap aprovado

Tabela 8.1 — As 15 etapas do UEF, com macro-fase, entregável principal e checkpoint de qualidade. Fonte: elaborado pelo autor com base em ISO 15288 e INCOSE Handbook.

◆ INSIGHT DO ENGENHEIRO

O UEF não é waterfall puro. Embora as 15 etapas sejam sequenciais, o framework admite iterações dentro de cada macro-fase. Em Concepção, requisitos podem ser revistos após análise de arquitetura. Em Design, escolha de sensores pode forçar revisão de plataforma. Em Implementação, piloto pode identificar problemas que exigem retorno ao protótipo. O importante é documentar cada iteração para manter rastreabilidade — princípio fundamental da ISO 15288.

8.3 Engenharia de Requisitos

Engenharia de requisitos é a disciplina que traduz necessidades de stakeholders em especificações técnicas precisas, testáveis e rastreáveis. É a etapa mais subestimada do ciclo de vida — e a que mais gera retrabalho quando mal executada. Esta seção analisa os seis componentes fundamentais da engenharia de requisitos para sistemas UNIIKER.

8.3.1 Requisitos Funcionais e Não Funcionais

Requisitos funcionais descrevem O QUE o sistema faz: medir temperatura ambiente a cada 5 minutos, detectar presença humana via PIR, publicar telemetria via MQTT, exibir temperatura atual no display IPS. São tipicamente expressos em formato Use Case (o sistema deve...). **Requisitos não funcionais** descrevem COMO o sistema faz, em termos de qualidade: latência inferior a 200 ms, autonomia mínima de 30 dias em bateria, disponibilidade 99,5%, consumo inferior a 100 mA médio. A ISO 25010 define 8 categorias de requisitos não funcionais: funcionalidade, confiabilidade, performance, eficiência, usabilidade, segurança, compatibilidade, manutenibilidade.

■ ENGENHARIA EM FOCO

Diferença crítica: requisitos funcionais são validados por testes de aceitação («o sistema faz X?»). Requisitos não funcionais são validados por testes de performance («o sistema faz X em menos de Y ms?»). Projetos que pulam requisitos não funcionais tipicamente descobrem tarde demais que «funciona» é diferente de «funciona dentro dos requisitos de qualidade». ISO 25010 deve ser checklist obrigatório.

8.3.2 Restrições, Premissas, Stakeholders e Critérios

Restrições são limites impostos externamente que não podem ser alterados: orçamento máximo, prazo máximo, regulamentações (ANATEL, ANVISA), tecnologias obrigatórias («deve usar AWS»). **Premissas** são hipóteses assumidas como verdadeiras sem validação: «usuários terão Wi-Fi disponível», «temperatura ambiente estará entre 0-50°C», «equipe terá 3 engenheiros durante 12 meses». **Stakeholders** são todas as partes interessadas: cliente, usuário final, patrocinador, equipe técnica, reguladores, sociedade. Cada stakeholder tem necessidades distintas que devem ser balanceadas. **Critérios de aceitação** são condições objetivas, testáveis, que definem quando um requisito está atendido: «temperatura medida com precisão de $\pm 0,5^{\circ}\text{C}$ no range 0-50°C, validada por 100 medições comparadas com termômetro calibrado».

▲ ERROS CLÁSSICOS

Erro Clássico #2: «Requisitos mudaram, vamos adaptar.»

Causa: ausência de processo formal de gestão de mudanças. Cada mudança ad-hoc invalida artefatos anteriores (arquitetura, código, testes) sem rastreabilidade.

Como evitar: implementar Change Control Board (CCB) que avalia cada mudança: impacto em custo, prazo, qualidade. Toda mudança aprovada atualiza baseline de requisitos. ISO 15288 seção 6.4.3 define processo formal.

Exemplo: cliente pede adicionar Bluetooth no projeto K10. Sem CCB, engenheiro adiciona, mas quebra battery life (BLE sempre ativo). Com CCB, mudança é analisada: impacto em autonomia, custo de retrabalho, etc.

8.4 Escolha da Plataforma

A escolha entre K10, M10 ou ambos é decisão arquitetural fundamental que afeta todo o ciclo de vida do projeto. Esta seção apresenta metodologia estruturada para essa decisão, baseada em árvore de decisão criteriosa.

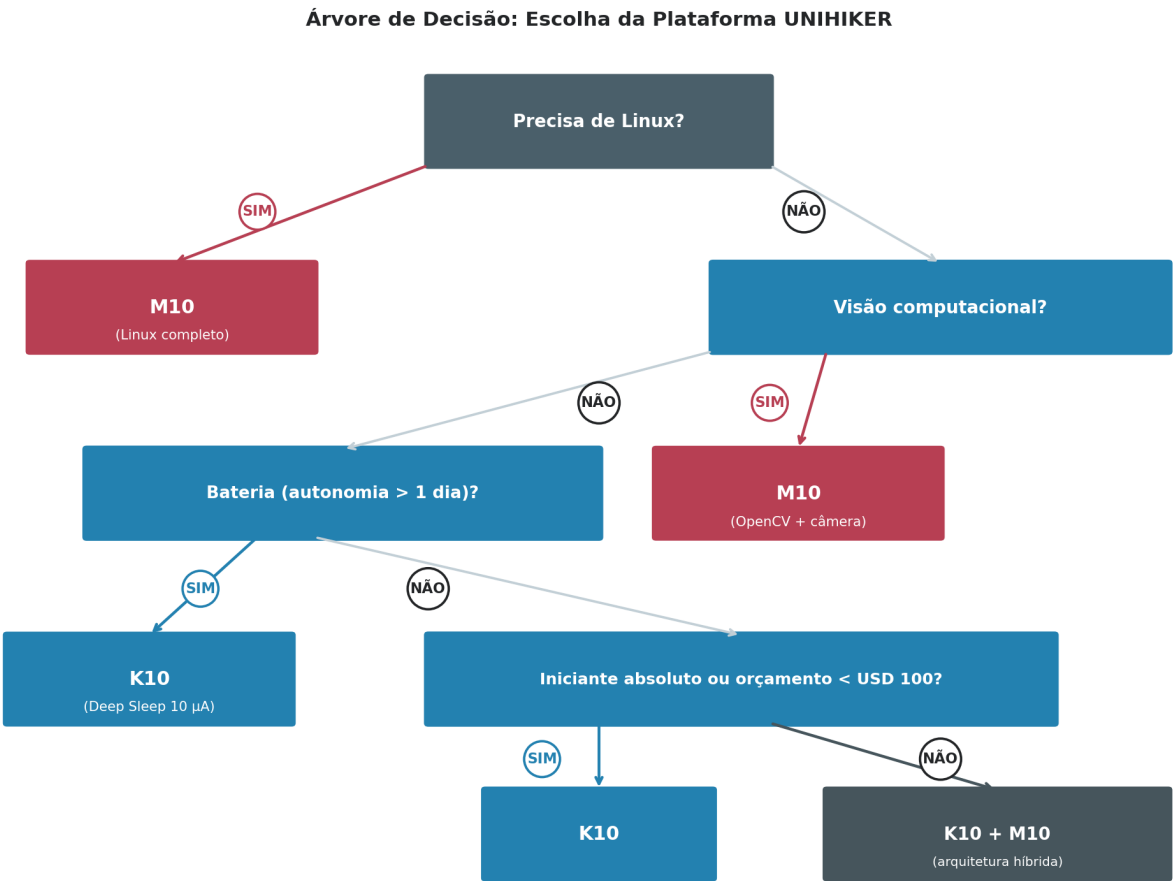


Figura 8.3 — Árvore de decisão para escolha da plataforma. Quatro perguntas sequenciais (Linux? Visão? Bateria? Iniciante?) conduzem à recomendação. Fonte: elaborado pelo autor.

8.4.1 Metodologia de Escolha

A árvore de decisão apresentada na Figura 8.3 é ponto de partida, mas a decisão final deve considerar três dimensões adicionais: **requisitos técnicos** (latência, RAM, IA), **requisitos operacionais** (autonomia, custo, equipe) e **requisitos estratégicos** (escala futura, parceiros, regulação). A Tabela 8.2 sintetiza quando escolher cada configuração.

Configuração	Quando Escolher	Quando Evitar
Apenas K10	Bateria + custo baixo + iniciante + sem Linux	Visão, IA grande, Linux, OpenCV
Apenas M10	Linux + visão + rede elétrica + equipe técnica	Bateria longa, custo baixo, iniciante
K10 + M10 (híbrida)	Escala > 10 dispositivos + Edge/Gateway/Cloud	Projeto único, sem escala, orçamento mínimo
Outra plataforma	Requisitos fora do escopo UNIHAKER (GPU, certificação SIL)	Quando UNIHAKER atende (não reinventar)

Tabela 8.2 — Configurações arquiteturais e quando escolher cada uma. Fonte: elaborado pelo autor.

❓ DECISÃO DO ENGENHEIRO

Cenário: Projeto de 1000 dispositivos IoT para monitoramento de máquinas industriais com IA preditiva.
Melhor escolha: arquitetura híbrida K10 + M10 + Cloud
Justificativa: K10 (1 por máquina) por custo + TinyML local. M10 (1 por linha de produção) por agregação + Edge AI. Cloud por histórico + ML batch. Escala justifica três camadas.
Alternativas descartadas: Apenas K10 (sem agregação local). Apenas M10 (1 GB RAM insuficiente para 1000 streams). Apenas Cloud (latência alta, custo de rede).

8.5 Seleção de Sensores

Seleção de sensores é decisão técnica com impacto direto em performance, custo e confiabilidade do sistema. Esta seção apresenta metodologia estruturada com oito critérios fundamentais.

Critério	Pergunta	Como Avaliar
Precisão	Qual erro máximo aceitável?	Datasheet + calibração laboratorial
Resolução	Qual menor variação detectável?	Bits do ADC + ruído
Consumo	Quanto consome em operação/sleep?	Datasheet + medição real
Calibração	Frequência e método?	Manual ou automática, com padrão rastreável
MTBF	Tempo médio entre falhas?	Datasheet + testes acelerados
Disponibilidade	Fornecedor tem estoque e lead time?	Cotação de 3+ fornecedores
Vida útil	Quanto dura em condições reais?	Testes de campo + feedback de outros projetos

Tabela 8.3 — Critérios para seleção de sensores. Fonte: elaborado pelo autor.

❓ TRADE-OFF DE ENGENHARIA

Trade-off clássico em sensores: precisão vs custo. Sensor de PM2.5 profissional (Plantower PMS5003, USD 30) tem precisão $\pm 10 \mu\text{g}/\text{m}^3$. Sensor de laboratório (TSI 8530, USD 5.000) tem $\pm 1 \mu\text{g}/\text{m}^3$. Para rede densa de 1000 sensores, PMS5003 permite cobertura 100x maior pelo mesmo custo. Para estação de referência regulatória, TSI é essencial. Decisão depende do caso de uso, não há vencedor universal.

8.6 Comunicação

Seleção de protocolos de comunicação segue critérios técnicos específicos: alcance, throughput, latência, consumo, custo e disponibilidade de infraestrutura. Esta seção sintetiza os critérios para os principais protocolos relevantes a sistemas UNIHAKER.

Protocolo	Alcance	Throughput	Consumo	Aplicação Típica
Wi-Fi	30-100 m	150 Mbps	Médio	Rede local estável
BLE 5.0	30-100 m	2 Mbps	Baixo	Wearables, pairing
LoRaWAN	2-15 km	0,3-50 kbps	Muito baixo	Sensores remotos
NB-IoT	10+ km	250 kbps	Baixo	Cellular IoT
Ethernet	100 m	100 Mbps	Médio	Indústria estável
Serial UART	15 m	1 Mbps	Baixo	MCU externo
MQTT	—	—	—	Telemetria IoT pub/sub
HTTP/REST	—	—	—	APIs web
CoAP	—	—	—	IoT em dispositivos restritos

Tabela 8.4 — Comparação de protocolos de comunicação. MQTT, HTTP e CoAP são protocolos de aplicação sobre os demais.
Fonte: elaborado pelo autor.

8.7 Arquitetura

Esta seção apresenta checklist arquitetural completo para sistemas UNIIKER, em oito dimensões que devem ser explicitamente endereçadas em qualquer projeto profissional.

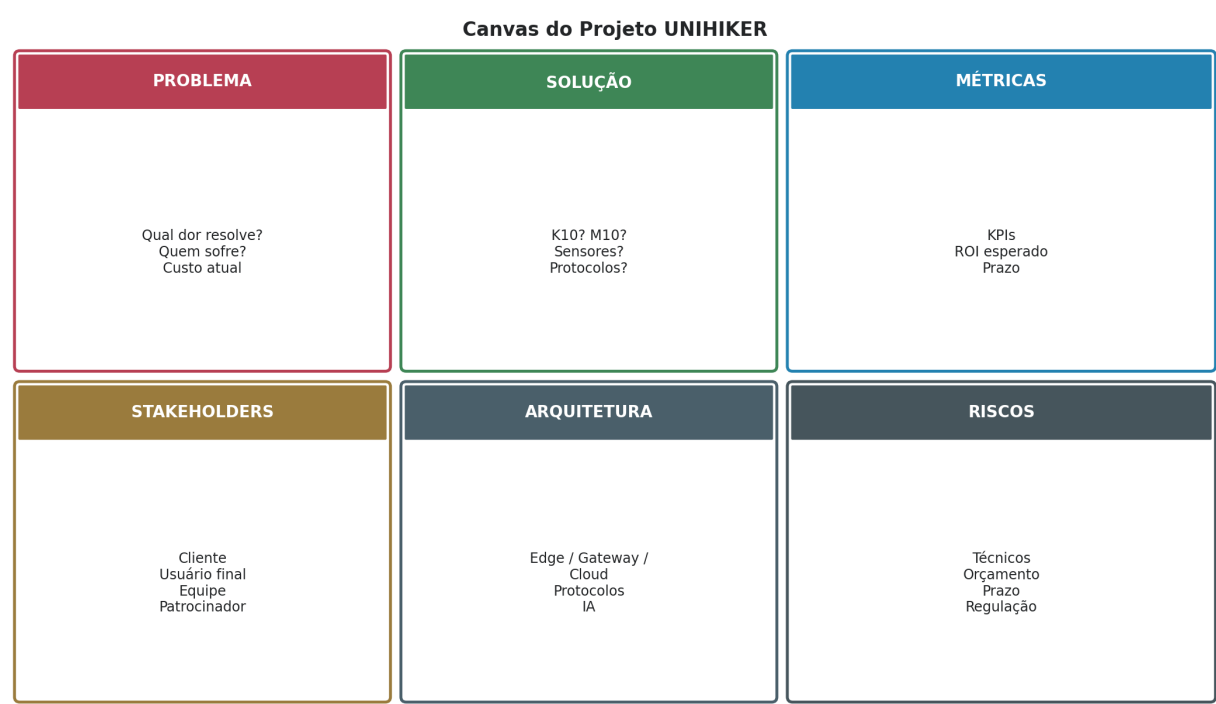


Figura 8.4 — Canvas do projeto UNIIKER em seis blocos: Problema, Solução, Métricas, Stakeholders, Arquitetura, Riscos. Fonte: elaborado pelo autor.

8.7.1 Checklist Arquitetural

Edge (K10): Sensores definidos, TinyML model selecionado, GPIO/pinos mapeados, consumo em Deep Sleep medido, autonomia calculada, case/proteção definida.

Gateway (M10): Linux version pinado, Docker compose definido, Node-RED flows, InfluxDB schema, OTA setup, monitoramento local.

Cloud: Provedor escolhido (AWS/Azure/GCP), região definida, compliance LGPD/HIPAA, custos estimados por 1000 dispositivos, redundância multi-region.

Dashboard: Ferramenta (Grafana/Looker/PowerBI), usuários e permissões, alertas configurados, data refresh rate, mobile-friendly.

Banco de Dados: Time-series (InfluxDB/Timestream) para telemetria, relacional (PostgreSQL/SQL Server) para configurações, retenção definida, backup automatizado.

Segurança: TLS em toda comunicação, autenticação (OAuth/JWT), autorização (RBAC), assinatura de firmware, secure boot, hardening do Linux.

OTA: Atualização remota de firmware K10 e M10, rollback automático em caso de falha, versionamento semântico, canário deploy (10% antes de 100%).

Escalabilidade: Plano para 10x crescimento, sharding de banco, cluster de brokers MQTT, CDN para dashboards, custo marginal por dispositivo.

8.8 Inteligência Artificial

Decisão sobre qual camada de IA usar (TinyML, Edge AI, Cloud AI, LLM, Agentes, IA Federada) tem impacto direto em latência, privacidade, custo e viabilidade técnica. Esta seção analisa quando usar cada abordagem.

Abordagem	Quando Usar	Quando Evitar
TinyML (K10)	Modelo <100 KB, latência <100 ms, bateria	Modelos complexos, visão, NLP
Edge AI (M10)	OpenCV, TFLite, MediaPipe, latência 100-500 ms	Modelos >100 MB, sem rede elétrica
Cloud AI	Modelos >1 GB, LLMs, treinamento, analytics	Latência crítica, offline, privacidade
LLM Local (M10)	Phi-3 2B quant, privacidade, offline, RAG local	Latência <500 ms, modelos >3B
Agentes	Multi-robôs, sistemas autônomos, swarm	Sistemas simples, single-device
IA Federada	Treino distribuído, privacidade de dados	Modelos pequenos, sem dispositivos suficientes

Tabela 8.5 — Quando usar cada abordagem de IA. Fonte: elaborado pelo autor.

8.9 Testes

Testes são a única defesa contra falhas em produção. Esta seção apresenta metodologia completa em seis níveis, organizada como pirâmide de testes.

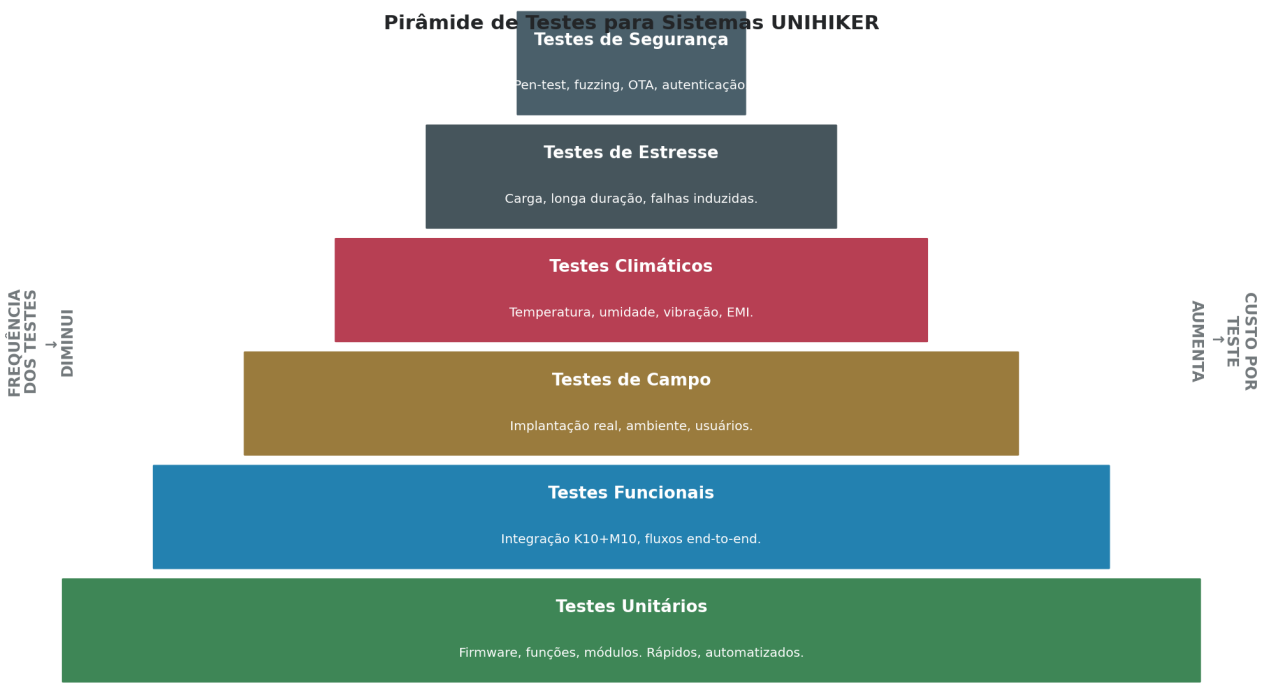


Figura 8.5 — Pirâmide de testes para sistemas UNIIKER, em seis níveis: unitários (base, frequentes, baratos) a segurança (topo, raros, caros). Fonte: elaborado pelo autor.

8.9.1 Níveis de Teste

Testes unitários validam funções e módulos individuais (firmware, bibliotecas Python). Automatizados, executam em CI/CD a cada commit. Cobertura mínima recomendada: 70%. **Testes funcionais** validam integração entre K10 e M10, fluxos end-to-end. Manuais ou automatizados via MQTT. **Testes de campo** validam implantação real em ambiente operativo. Mínimo 90 dias de coleta de dados antes do rollout. **Testes climáticos** validam operação em temperatura (-20 a +70°C), umidade (10-95%), vibração, EMI. Câmara térmica ou câmara ambiental. **Testes de estresse** validam carga extrema (100x tráfego normal), longa duração (30+ dias contínuos), falhas induzidas (corte de energia, rede). **Testes de segurança** validam resistência a ataques: pen-test, fuzzing, OTA assinado, brute force.

▲ CRITÉRIO DE APROVAÇÃO

Quando um projeto pode sair do laboratório?

Critérios mínimos para rollout de produção:

- ✓ 90+ dias de operação contínua sem crash
- ✓ Testes climáticos aprovados (temperatura, umidade)
- ✓ Testes de campo em 3+ locais distintos
- ✓ Pen-test de segurança executado por terceiro
- ✓ OTA testado com rollback funcionando
- ✓ Documentação técnica completa (arquitetura, API, ops)
- ✓ Plano de manutenção preventiva definido
- ✓ Equipe de suporte treinada (mínimo 2 pessoas)
- ✓ SLA definido e monitoramento 24/7 ativo

Sem esses critérios, projeto NÃO está pronto para produção.

8.10 Implantação

Implantação é a transição do laboratório para o ambiente operativo. Esta seção analisa as cinco sub-fases: piloto, rollout, treinamento, documentação e suporte.

8.10.1 Piloto e Rollout

Piloto é implantação controlada em poucos sites (3-10), com coleta intensiva de métricas e feedback. Duração típica: 90 dias. Critério de sucesso: 99% uptime, 0 incidentes críticos, NPS de usuários acima de 7/10. **Rollout** é expansão para todos os sites, em fases (10% → 30% → 70% → 100%), com período de observação entre fases (mínimo 2 semanas). Em caso de falha em qualquer fase, rollback para fase anterior. Canário deploy: implantar em 5% primeiro, monitorar 24-48h, expandir se estável.

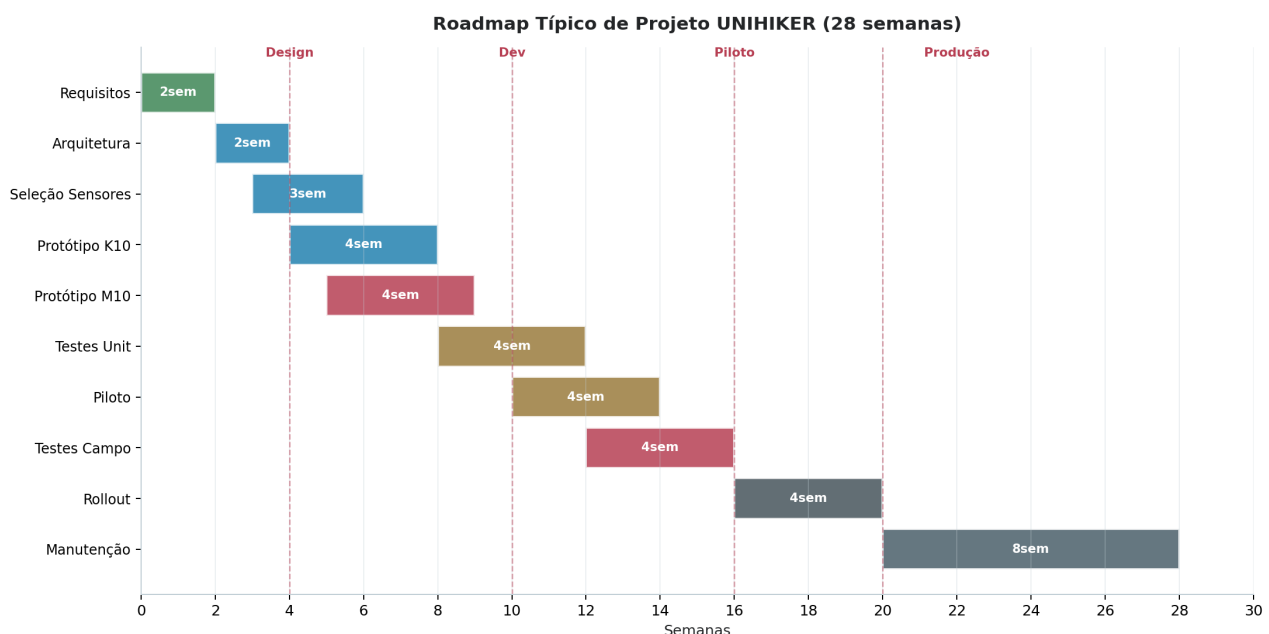


Figura 8.6 — Roadmap típico de projeto UNIHAKER em 28 semanas, com fases: Design (0-4sem), Dev (4-10sem), Piloto (10-16sem), Produção (16-28sem). Fonte: elaborado pelo autor.

8.11 Operação

Operação é a fase mais longa do ciclo de vida — pode durar anos após o rollout. Esta seção analisa os sete componentes fundamentais: monitoramento, logs, alertas, OTA, atualizações, backups e segurança.

8.11.1 Componentes de Operação

Monitoramento contínuo de métricas técnicas (CPU, RAM, rede) e de negócio (dispositivos online, telemetria recebida). Ferramentas: Prometheus + Grafana, Datadog, CloudWatch. **Logs** centralizados com ELK stack (Elasticsearch, Logstash, Kibana) ou Loki. Retenção mínima 90 dias para auditoria. **Alertas** classificados por severidade (P1 crítico, P2 alto, P3 médio, P4 baixo). P1 exige resposta em 15 min, P4 em 24h. Integração com PagerDuty, Opsgenie. **OTA** permite atualizar firmware remotamente. Versão canário (5%) antes de rollout 100%. Rollback automático em caso de falha.

▲ CHECKLIST DO ENGENHEIRO

Projeto pronto para operação?

- ☐ Monitoramento ativo 24/7 com alertas
- ☐ Logs centralizados com retenção 90+ dias
- ☐ OTA testado com rollback funcionando
- ☐ Backups automáticos testados (restauração validada)
- ☐ Plano de resposta a incidentes documentado
- ☐ Equipe de on-call definida (rotação 24/7)
- ☐ Runbooks para incidentes comuns
- ☐ Comunicação com cliente em caso de outage
- ☐ SLA definido e monitorado
- ☐ Post-mortem após cada incidente P1/P2

8.12 Manutenção

Manutenção é a disciplina que mantém o sistema operante após o rollout. Quatro tipos de manutenção coexistem: preventiva, corretiva, preditiva e evolutiva.

Tipo	Quando	Exemplo	Custo
Preventiva	Programada, antes de falhas	Calibração de sensores trimestral	Baixo
Corretiva	Reativa, após falha	Substituir K10 que parou de responder	Médio-alto
Preditiva	Baseada em IA, prevê falha	Trocar bearing de motor antes quebrar	Médio (requer ML)

Tipo	Quando	Exemplo	Custo
Evolutiva	Adiciona features	Novo dashboard, novo sensor	Variável

Tabela 8.6 — Quatro tipos de manutenção em sistemas UNIIKER. Fonte: elaborado pelo autor.

8.13 Escalabilidade

Escalabilidade é a capacidade do sistema crescer em número de dispositivos sem degradação. Esta seção analisa os seis níveis típicos de crescimento.

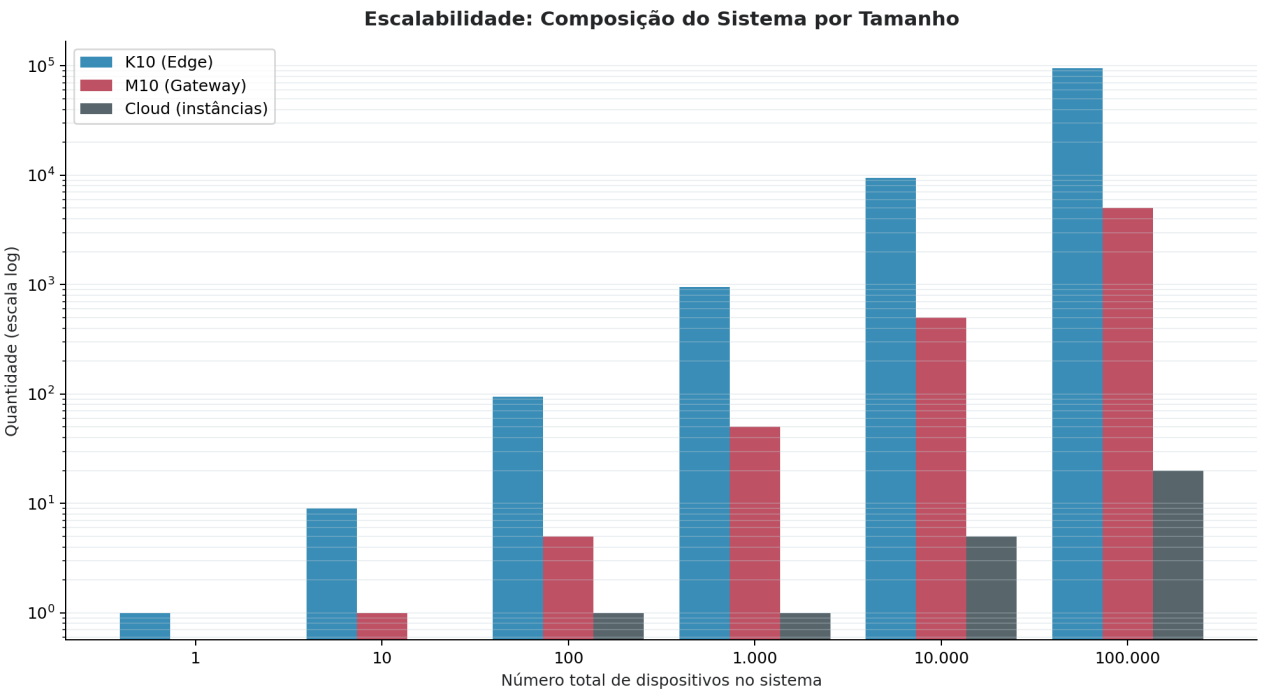


Figura 8.7 — Escalabilidade: composição do sistema (K10, M10, Cloud) por tamanho total do sistema, em escala logarítmica. Fonte: elaborado pelo autor.

Escala	K10	M10	Cloud	Desafio Principal
1	1	0	0	Desenvolvimento
10	9	1	0	Configuração manual
100	95	5	1	Wi-Fi congestionado
1.000	950	50	1	Broker MQTT único
10.000	9.500	500	5	Storage + governança
100.000	95.000	5.000	20	Custo cloud, multi-region

Tabela 8.7 — Escalabilidade em seis níveis, com desafio principal em cada um. Fonte: elaborado pelo autor.

▲ ERROS CLÁSSICOS

Erro Clássico #3: «Funciona com 10, então funciona com 10.000.»

Causa: arquitetura linear sem considerar gargalos em escala.

Gargalos típicos: (1) broker MQTT único sem cluster; (2) banco de dados sem sharding; (3) Wi-Fi congestionado; (4) custo de cloud exponencial; (5) ausência de cache.

Como evitar: projetar para 10x a escala alvo desde o início. Testar com 1000+ dispositivos em simulação antes do rollout.

Exemplo: sistema de smart city projetado para 100 dispositivos, depois expandido para 10.000. Cloud original (single broker, single DB) colapsou. Solução: redesign completo da camada cloud.

8.14 Estudos de Caso

Esta seção apresenta sete estudos de caso que aplicam o UEF em domínios distintos. Cada estudo demonstra como as 15 etapas do framework se materializam em contexto real.

8.14.1 ETA (Estação de Tratamento de Água)

Aplicação do UEF em ETA municipal: requisitos regulatórios ANVISA, arquitetura K10 (Modbus RTU) + M10 (Node-RED) + Azure (compliance), piloto em 1 ETA, rollout para 50 ETAs da companhia estadual. Manutenção preventiva trimestral de sensores. Escalabilidade para 500+ ETAs em 3 anos.

8.14.2 ETE (Estação de Tratamento de Esgoto)

Similar à ETA, com desafios adicionais de ambiente agressivo (H2S corrosivo) e conformidade IBAMA. Sensores com encapsulamento IP68. K10 com Modbus + LoRa. M10 com InfluxDB + Node-RED. Cloud AWS para relatórios ambientais.

8.14.3 Agricultura de Precisão

UEF aplicado em fazenda 1000 hectares: K10 solar por talhão (LoRaWAN), M10 regional com visão (câmera MIPI para pragas), AWS + SageMaker para previsão de safra. Piloto em 50 hectares, escala para 1000+ em 2 anos.

8.14.4 Hospital Inteligente

UEF em hospital: K10 wearable por paciente (BLE), M10 por ala (Node-RED + alertas), Azure IoT (LGPD/HIPAA). Testes rigorosos (ANVISA), piloto em 1 enfermaria, rollout para 10 alas. Manutenção corretiva 24/7.

8.14.5 Smart City

UEF em escala metropolitana: K10 em postes (LoRaWAN), M10 por bairro, Google Cloud multi-region. Governança inter-secretarial complexa. Piloto em 1 bairro, expansão para 1500 bairros. Custo total USD 5M+.

8.14.6 Universidade

UEF em smart campus: K10 por sala (Wi-Fi), M10 por prédio, Firebase. Projeto acadêmico com alunos de engenharia. Escala para 500 salas. Manutenção por próprios alunos como parte de disciplina.

8.14.7 Amazônia

UEF em monitoramento ambiental: K10 solar + LoRaWAN, M10 em torres com 4G/satélite, AWS + Rekognition. Logística complexa, manutenção por barco/avião. Escala para 600+ UCs.

8.15 Checklist Final

Esta seção consolida em um único checklist profissional todos os itens que devem ser verificados antes da implantação de qualquer sistema UNIHIKER em produção.

Checklist Pré-Implantação

✓

Hardware montado e testado

✓

Sensores calibrados e validados

✓

Firmware estável (sem crashes 7+ dias)

✓

IA validada (acurácia medida)

✓

Dashboard funcional e intuitivo

✓

OTA implementado e testado

✓

Segurança (TLS, auth, assinatura)

✓

Backup automático configurado

✓

Testes de campo concluídos

✓

Documentação técnica completa

✓

Operação 24/7 monitorada

✓

Plano de manutenção definido

✓

Equipe treinada

✓

Plano de rollback documentado

✓

Comunicação com cliente validada

Figura 8.8 — Checklist visual pré-implantação em 15 itens essenciais. Fonte: elaborado pelo autor.

▲ CHECKLIST DO ENGENHEIRO**CHECKLIST PRÉ-IMPLANTAÇÃO — PARTE 1: Hardware, Firmware e IA****Hardware:**

- ☐ Placas K10/M10 testadas individualmente
- ☐ Sensores calibrados (rastreadabilidade NIST/INMETRO)
- ☐ Case/proteção IP definido para ambiente
- ☐ Fonte de alimentação dimensionada (margem 30%)

Firmware:

- ☐ Versão estável (sem crashes 7+ dias)
- ☐ OTA testado com rollback funcionando
- ☐ Watchdog configurado
- ☐ Logging estruturado

IA:

- ☐ Modelo validado (acurácia medida em dataset real)
- ☐ Latência de inferência medida
- ☐ Plano de retreino definido

▲ CHECKLIST DO ENGENHEIRO**CHECKLIST PRÉ-IMPLANTAÇÃO — PARTE 2: Comunicação, Dashboard e Segurança****Comunicação:**

- ☐ TLS em toda comunicação
- ☐ Broker MQTT com cluster/HA
- ☐ Fallback de conectividade

Dashboard:

- ☐ Grafana/Looker funcional
- ☐ Alertas configurados (P1-P4)
- ☐ Mobile-friendly

Segurança:

- ☐ Pen-test executado por terceiro
- ☐ Autenticação (OAuth/JWT)
- ☐ Autorização (RBAC)
- ☐ Secure boot configurado

▲ CHECKLIST DO ENGENHEIRO

CHECKLIST PRÉ-IMPLANTAÇÃO — PARTE 3: Documentação, Backups, Testes, Operação e Manutenção

Documentação: arquitetura, API (OpenAPI), runbooks, manual do usuário

Backups: automático testado, restauração validada, retenção 90+ dias

Testes: unitários (70%+ cobertura), campo 90+ dias, climáticos, pen-test

Operação: monitoramento 24/7, equipe on-call, SLA definido

Manutenção: plano preventivo, fornecedor de peças, MTTR estimado

8.15.1 Matriz RACI

Para projetos multi-pessoa, matriz RACI (Responsible, Accountable, Consulted, Informed) clarifica responsabilidades por atividade. Figura 8.9 apresenta matriz típica para projeto UNIHAKER.

Matriz RACI por Atividade do Projeto							
	Gerente Projeto	Arquiteto	Eng. Firmware	Eng. Hardware	Eng. DevOps	QA	Cliente/ Stakeholder
Levantamento Requisitos	A	C	I	I	I	I	R
Arquitetura	A	R	C	C	C	I	I
Seleção Sensores	I	A	C	R	I	I	C
Protótipo	A	C	R	R	C	C	I
Testes	A	C	C	C	C	R	I
Piloto	A	C	R	C	C	C	R
Rollout	A	C	C	C	R	C	I
Manutenção	A	I	R	C	R	C	I
Evolução	A	R	C	C	C	I	C

R = Responsável

A = Aprovador

C = Consultado

I = Informado

Figura 8.9 — Matriz RACI por atividade do projeto, com 7 papéis e 9 atividades. R=Responsável, A=Aprovador, C=Consultado, I=Informado. Fonte: elaborado pelo autor.

Conclusão do Capítulo

O UNIHAKER Engineering Framework (UEF) apresentado neste capítulo é mais que uma metodologia: é um framework mental que estrutura o pensamento de engenheiros e arquitetos diante de qualquer projeto de sistema inteligente. As 15 etapas, agrupadas em 4 macro-fases, cobrem desde a concepção da ideia até a evolução do produto em produção. As quatro novas caixas editoriais — Checklist do Engenheiro, Critério de Aprovação, Erros Clássicos e Visão do Consultor — sintetizam, em formato imediatamente acionável, décadas

de experiência consolidada em engenharia de sistemas.

▲ VISÃO DO CONSULTOR

Visão do Consultor — mensagem final: uma empresa que adota o UEF ganha três vantagens competitivas. Primeiro, **reduz risco**: framework estruturado antecipa problemas que projetos ad-hoc descobrem tarde demais. Segundo, **acelera entrega**: etapas bem definidas eliminam retrabalho e ambiguidade. Terceiro, **facilita auditoria**: artefatos documentados em cada etapa permitem rastreabilidade exigida por normas ISO 15288 e ISO 25010. Empresas que pulam metodologia «para economizar tempo» tipicamente gastam 3-5x mais tempo total — e ainda entregam sistemas frágeis. O UEF é investimento, não custo.

Para o leitor que chegou até aqui, a mensagem final é: o UEF não garante sucesso — mas diminui drasticamente a probabilidade de fracasso. Projetos de sistemas inteligentes são complexos demais para serem conduzidos intuitivamente. Metodologia é o que separa amadores de profissionais. Os capítulos subsequentes aprofundarão implementação prática de projetos selecionados, sempre sobre a base metodológica aqui estabelecida. Dominar o UEF é pré-requisito para conduzir projetos UNIHAKER em escala profissional.

Referências Bibliográficas

As referências a seguir seguem o padrão ABNT NBR 6023:2018.

- DFROBOT. **UNIHAKER K10 and M10 product documentation**. Shanghai: DFRobot, 2024. Disponível em: <https://wiki.unihiker.com>. Acesso em: 15 fev. 2025.
- IEEE. **IEEE Standard for System and Software Engineering — System Life Cycle Processes**. IEEE Std 15288-2015. New York: IEEE, 2015.
- INCOSE. **INCOSE Systems Engineering Handbook: A guide for system life cycle processes and activities**. 4. ed. Hoboken: Wiley, 2015.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 15288: Systems and software engineering — System life cycle processes**. Geneva: ISO, 2015.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models**. Geneva: ISO, 2011.
- PROJECT MANAGEMENT INSTITUTE. **A guide to the project management body of knowledge (PMBOK guide)**. 7. ed. Newton Square: PMI, 2021.
- PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.
- BROY, Manfred; et al. Engineering safe software systems. **Journal of Systems and Software**, v. 86, n. 1, p. 1-25, jan. 2013. DOI: 10.1016/j.jss.2012.09.014.
- BOEHM, Barry W. **Software engineering economics**. Englewood Cliffs: Prentice-Hall, 1981.
- RECHTIN, Eberhardt; MAIER, Mark W. **The art of systems architecting**. 2. ed. Boca Raton: CRC Press, 2010.
- LINUX FOUNDATION. **Edge computing reference architecture**. San Francisco: Linux Foundation, 2023. Disponível em: <https://www.lfedge.org>. Acesso em: 15 fev. 2025.
- WARDEN, Pete; SITUNAYAKE, Daniel. **TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers**. Sebastopol: O'Reilly Media, 2020.

- KERZNER, Harold. **Project management: a systems approach to planning, scheduling, and controlling**. 12. ed. Hoboken: Wiley, 2017.
- HIGHSMITH, Jim. **Agile project management: creating innovative products**. 2. ed. Boston: Addison-Wesley, 2009.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.
- RIES, Eric. **The lean startup: how today's entrepreneurs use continuous innovation to create radically successful businesses**. New York: Crown Business, 2011.
- BROOKS, Frederick P. **The mythical man-month: essays on software engineering**. Anniversary ed. Boston: Addison-Wesley, 1995.
- ZHANG, Wei; LI, Hao; CHEN, Jun. A comparative study of single-board computers for educational edge AI. **IEEE Transactions on Education**, v. 67, n. 2, p. 112-125, maio 2024. DOI: 10.1109/TE.2024.3359210.
- TANENBAUM, Andrew S.; BOS, Herbert. **Modern operating systems**. 4. ed. Boston: Pearson, 2015.